HiDPI Support & Rendering in Cute Framework

A technical review of high-DPI handling, with focus on shape and text rendering

Date: 2026-07-05 · Scope: `src/cute_app.cpp`, `src/cute_input.cpp`, `src/cute_draw.cpp`, `src/cute_graphics_sdlgpu.cpp`, `src/cute_graphics_gles.cpp`, `include/cute_app.h`

---

1. Executive summary

Cute Framework (CF) *enables* high-DPI at the OS/window level but does not *honor* it in its rendering pipeline. The framework unconditionally requests a high-pixel-density backbuffer, yet all of its internal rendering — shapes, text, sprites — happens on an **offscreen canvas sized in logical points**, which is then **upscaled to the physical framebuffer** with a single stretch blit.

The practical result on any HiDPI display (Apple Retina, Windows/Linux fractional scaling, most modern phones/tablets):

- **Everything is rendered at low resolution and then magnified.** A 2× Retina display throws away ~75% of its available resolution.
- **Text is blurry.** Glyphs are rasterized by `stb_truetype` at the *logical* font size and then stretched, so a 20 px font is rasterized at 20 px and displayed across ~40 physical pixels.
- **Shape anti-aliasing is done in the wrong space.** The SDF anti-alias edge is computed as "one logical pixel" and then magnified, so edges are ~2 physical pixels of blur instead of a crisp 1-pixel transition.
- **The public API is self-contradictory** about whether sizes are in "pixels" or "points," and the exposed `dpi_scale` value is informational only — nothing in the renderer consumes it.

None of this is catastrophic — CF apps *run* correctly and are fully interactive — but on HiDPI hardware they look noticeably soft compared to native or DPI-aware peers. This report documents the root causes, the specific bugs, and a staged plan to fix them.

---

2. How CF sets up the window and canvas today

2.1 High-DPI is force-enabled

At window creation CF always sets the SDL high-pixel-density flag:

```
// src/cute_app.cpp:284
flags |= SDL_WINDOW_HIGH_PIXEL_DENSITY; // Turn on high DPI support for all platforms.
```

This means SDL gives the window a backbuffer whose **pixel** size is `logical_size × pixel_density`. On a 2× Retina display, a window requested as 640×480 *points* gets a 1280×960 *pixel* backbuffer.

2.2 The DPI scale that CF stores is a *display* scale, not the backbuffer ratio

```
// src/cute_app.cpp:320
app->dpi_scale = SDL_GetWindowDisplayScale(app->window);
```

`SDL_GetWindowDisplayScale()` returns the OS's *suggested content/UI scale* for the display the window is on. This is **not** the same quantity as `SDL_GetWindowPixelDensity()`, which is the actual ratio between the pixel backbuffer and the logical window size. On integer-scaled macOS Retina they usually coincide (both 2.0), but on **Windows at 150%** or **Wayland fractional scaling** they can diverge. CF stores and exposes the display scale but would need the *pixel density* to correctly size a framebuffer — so even the one DPI number CF exposes is the wrong one for rendering purposes.

2.3 The offscreen canvas is created in *logical* units

All CF drawing is routed through an offscreen canvas (`app->offscreen_canvas`). It is created with the logical window dimensions:

```
// src/cute_app.cpp:155
static void s_canvas(int w, int h)
{
    CF_CanvasParams params = cf_canvas_defaults(w, h);
    params.target.filter = CF_FILTER_LINEAR;
    ...
    app->offscreen_canvas = cf_make_canvas(params);
    app->canvas_w = w;
    app->canvas_h = h;
}

// src/cute_app.cpp:339 (inside cf_make_app)
s_canvas(app->w, app->h);   // app->w/app->h are the *logical* window size
```

`app->w` / `app->h` are seeded from the requested window size and are subsequently updated from **logical** window-resize events (see §2.5). `dpi_scale` never enters this calculation. So the offscreen canvas is always at logical resolution regardless of the display.

2.4 The frame is rendered to the logical canvas, then stretched to the physical backbuffer

```
// src/cute_app.cpp:527
cf_render_to(app->offscreen_canvas, clear);      // draw everything at logical res
...
cf_sdlgpu_blit_canvas(app->offscreen_canvas);    // stretch onto the physical swapchain
```

The SDL_GPU blit stretches the logical-sized canvas over the full physical swapchain texture:

```
// src/cute_graphics_sdlgpu.cpp:855
// Stretch the app canvas onto the backbuffer canvas.
SDL_GPUBlitRegion src = { .w = canvas_internal->w, .h = canvas_internal->h };   // logical
SDL_GPUBlitRegion dst = { .w = g_ctx.swapchain_tex_w, .h = g_ctx.swapchain_tex_h }; // physical
SDL_GPUBlitInfo blit_info = {
    .source = src, .destination = dst,
    .filter = SDL_GPU_FILTER_NEAREST,   // <-- nearest-neighbour upscale
    ...
};
SDL_BlitGPUTexture(g_ctx.cmd, &blit_info);
```

The GLES backend does the same thing, but with a **linear** filter:

```
// src/cute_graphics_gles.cpp:717
SDL_GetWindowSizeInPixels(g_ctx.window, &window_width, &window_height); // physical
glBlitFramebuffer(
    0, canvas->h, canvas->w, 0,          // src: logical canvas
    0, 0, window_width, window_height,   // dst: physical window
    GL_COLOR_BUFFER_BIT, GL_LINEAR);     // <-- linear upscale
```

This is the crux of the whole issue: **the render target that all shapes and text land on is logical-sized, and the only step that touches physical resolution is a dumb stretch.** No amount of extra physical pixels can add detail that was never rendered.

2.5 Resize path also works in logical units

```
// src/cute_input.cpp:517
case SDL_EVENT_WINDOW_RESIZED:
    app->window_state.resized = true;
    app->w = event.window.data1;   // logical points, not pixels
    app->h = event.window.data2;
    break;
```

`SDL_EVENT_WINDOW_RESIZED` reports **logical** window coordinates. Note also that this handler updates `app->w/h` but does **not** resize the offscreen canvas — the canvas keeps whatever size it had at creation (CF uses a fixed-internal-resolution model). Combined with §2.1 this guarantees a non-1:1 blit on HiDPI.

2.6 DPI-change events update the number but nothing acts on it

```
// src/cute_input.cpp:557
case SDL_EVENT_WINDOW_DISPLAY_SCALE_CHANGED:
    app->dpi_scale = SDL_GetWindowDisplayScale(app->window);
    app->dpi_scale_was_changed = true;
    break;
```

`cf_app_dpi_scale_was_changed()` lets the *user* react (e.g. reload assets), but CF itself does not resize the canvas, re-rasterize fonts, or otherwise respond. Moving a window between a 1× and a 2× monitor changes the flag but not the render quality.

---

3. Text rendering under HiDPI

Text is where the blur is most visible, because glyph rasterization is resolution-dependent.

3.1 Glyphs are rasterized at the logical font size

```
// src/cute_draw.cpp:1936
static void s_render(CF_Font* font, CF_Glyph* glyph, float font_size, int blur)
{
    ...
    float scale = stbtt_ScaleForPixelHeight(&font->info, font_size);   // font_size is logical
    stbtt_GetGlyphBitmapBox(&font->info, glyph->index, scale, scale, &x0, &y0, &x1, &y1);
    int w = x1 - x0 + pad*2;
    int h = y1 - y0 + pad*2;
    ...
    stbtt_MakeGlyphBitmap(&font->info, ..., w - pad*2, h - pad*2, w, scale, scale, glyph->index);
}
```

The glyph bitmap has exactly `font_size`-tall coverage. The glyph sprite is then placed into the world 1:1 (`glyph->q0/q1` are in the same logical units as the font size, see `src/cute_draw.cpp:2759`). Because the whole canvas is later magnified by `dpi_scale`, that 20-px bitmap is displayed over ~40 physical pixels → **soft, muddy text**.

3.2 The glyph cache key has no DPI dimension

```
// src/cute_draw.cpp:1845
CF_INLINE uint64_t cf_glyph_key(int cp, float font_size, int blur)
{
    ...
    int k1 = (int)(font_size * 1000.0f);
    int k2 = blur;
    ...
}
```

The cache is keyed on `(codepoint, font_size, blur)`. There is no notion of "render this glyph at 2× for a Retina panel." Any DPI-aware fix must factor the scale into either this key or the `font_size` passed in.

3.3 Consequence

Even if the offscreen canvas were made physical-sized (§5.1), text would still be blurry unless glyphs are **rasterized at `font_size × dpi_scale`**. The two fixes are coupled: crisp text needs (a) a physical-resolution target *and* (b) DPI-scaled glyph rasterization with a DPI-aware cache key.

---

4. Shape rendering under HiDPI

CF draws shapes (quads, circles, capsules, triangles, lines) as signed-distance-field (SDF) primitives with analytic anti-aliasing. The anti-alias width is expressed in world units via an "anti-alias factor" (`aaf`):

```
// src/cute_draw.cpp:448
// Sets the anti-alias factor, the width of roughly one pixel scaled.
void CF_Draw::set_aaf()
{
    float inv_cam_scale = 1.0f / len(s_draw->cam_stack.last().m.y);
    float scale = s_draw->antialias.last();
    aaf = scale * inv_cam_scale;   // "one pixel" == one *logical* canvas pixel
}
```

`aaf` is "roughly one pixel," but that pixel is a **logical canvas pixel**, because the canvas is logical-sized. After the DPI upscale, that 1-pixel-wide gradient edge is smeared across `dpi_scale` physical pixels. Two things follow:

1. **Edges are softer than they should be.** A correct HiDPI renderer would anti-alias over one *physical* pixel, producing a crisp 1-pixel transition. CF produces a `dpi_scale`-pixel transition.

2. **The stretch blit adds a second layer of blur.** The SDF gradient is already a smooth ramp; magnifying it (nearest on SDL_GPU, linear on GLES) doesn't "re-alias" it to the physical grid, it just enlarges the ramp. So shape outlines look fat and fuzzy rather than sharp.

The projection is likewise logical:

```
// src/cute_draw.cpp:460
s_draw->projection = ortho_2d(0, 0, (float)app->w, (float)app->h);
```

So the entire shape pipeline lives in logical space and inherits the same magnification penalty as text. Thin strokes (`thickness` of 1) are the worst case: a 1-unit stroke is 1 logical pixel, drawn at low resolution, then blown up.

---

5. Backend inconsistencies & smaller issues

#	Issue	Location	Impact
B1	Upscale filter differs by backend: **NEAREST** on SDL_GPU vs **LINEAR** on GLES	`cute_graphics_sdlgpu.cpp:874`, `cute_graphics_gles.cpp:736`	Same app looks different (blocky vs blurry) depending on backend. Neither is "correct" — the real fix removes the upscale.
B2	`dpi_scale` uses `SDL_GetWindowDisplayScale` (content scale) not `SDL_GetWindowPixelDensity` (backbuffer ratio)	`cute_app.cpp:320`, `cute_input.cpp:558`	The exposed number can disagree with the actual framebuffer size on fractional-scaling platforms.
B3	API doc says window size is "in pixels," but the values are logical points on HiDPI	`include/cute_app.h:367`, `:377`, `:385`	Misleads users doing their own pixel-space math; latent because points==pixels at 1×.
B4	Touch coordinates scaled by `app->w/h` with acknowledged uncertainty	`cute_input.cpp:674-691` (`// NOTE: Probably wrong for high-DPI.`)	Touch mapping may be off on HiDPI mobile; already flagged in-source.
B5	Docs advise "most of the time you should ignore dpi" and there is no worked example of a DPI-aware app	`include/cute_app.h:399-409`	Users have no guidance and no clean path to sharp rendering.
B6	Offscreen canvas is not resized on `SDL_EVENT_WINDOW_RESIZED`, only on explicit calls	`cute_input.cpp:517`, `cute_app.cpp:751`	Intentional (fixed-resolution model), but interacts badly with the forced HiDPI backbuffer.

6. Root-cause summary

Everything above reduces to **one architectural decision**:

> CF renders into a logical-sized offscreen canvas and stretches it to a forced high-density backbuffer, while `dpi_scale` is computed but never fed back into canvas sizing, font rasterization, or the anti-alias factor.

Fix the target resolution and the two dependent quantities (glyph raster size, `aaf`), and both text and shapes become crisp with no change to user-facing coordinates.

7. Recommended fixes

The guiding principle: **let users keep working in logical/point coordinates, but render at physical resolution under the hood.** This preserves CF's simple mental model while producing sharp output.

7.1 Fix 1 — Render the app canvas at physical resolution *(highest impact)*

Make the offscreen canvas match the physical backbuffer, and expose a separate "logical size" for user-facing coordinate math.

- Track the real backbuffer ratio with `SDL_GetWindowPixelDensity()` (call it `pixel_scale`), separate from the informational `dpi_scale`.
- Size the offscreen canvas as `logical_w × pixel_scale`, `logical_h × pixel_scale`.
- Keep the default camera/projection mapping **logical** units so existing games don't move — i.e. scale the projection so that 1 logical unit spans `pixel_scale` device pixels.
- The final blit becomes 1:1 (or is dropped in favour of rendering straight to the swapchain), so the upscale blur disappears entirely.

Sketch:

```
// On create and on WINDOW_RESIZED / DISPLAY_SCALE_CHANGED:
float pixel_scale = SDL_GetWindowPixelDensity(app->window);
int px_w = (int)(app->w * pixel_scale);
int px_h = (int)(app->h * pixel_scale);
s_canvas(px_w, px_h);            // canvas is now physical-sized
app->canvas_pixel_scale = pixel_scale;
```

Users who deliberately want a retro low-res look can still call `cf_app_set_canvas_size()` to pin a small canvas — the point-sampled upscale then becomes a *feature* rather than an accident.

7.2 Fix 2 — Rasterize glyphs at physical size *(coupled with Fix 1)*

- Multiply the rasterization size by the active pixel scale: rasterize at `font_size × pixel_scale`.
- Add the scale to the cache key so 1× and 2× glyphs don't collide:

```
CF_INLINE uint64_t cf_glyph_key(int cp, float font_size, int blur, float pixel_scale) {
    int k0 = cp;
    int k1 = (int)(font_size * pixel_scale * 1000.0f);
    int k2 = blur;
    ...
}
```

- Draw the higher-resolution glyph bitmap scaled *down* into the same logical quad, so text metrics and layout are unchanged but the on-screen result is sharp.
- Re-render the glyph cache (or let it repopulate lazily) on `dpi_scale_was_changed`.

7.3 Fix 3 — Anti-alias in physical-pixel space

Once the canvas is physical-sized, `aaf` should represent **one physical pixel**. Divide by `pixel_scale` so the SDF edge is a crisp single device-pixel ramp:

```
void CF_Draw::set_aaf() {
    float inv_cam_scale = 1.0f / len(cam_stack.last().m.y);
    aaf = antialias.last() * inv_cam_scale / app->canvas_pixel_scale;
}
```

7.4 Fix 4 — Correct the DPI API & documentation

- Distinguish two concepts publicly:
  - `cf_app_get_dpi_scale()` — OS content/UI scale (keep, for users who want to scale UI).
  - `cf_app_get_pixel_scale()` *(new)* — backbuffer-to-logical ratio, the value the renderer uses.
- Fix the header docs on `cf_app_get_size/width/height` to say **logical points** (or add pixel-size accessors) so `include/cute_app.h:367` etc. match reality.
- Resolve the touch-coordinate `NOTE: Probably wrong for high-DPI` (`cute_input.cpp:674-691`) now that a defined pixel/point relationship exists.

7.5 Fix 5 — Make the two backends consistent

Whatever residual upscale remains (e.g. for the intentional low-res-canvas case), pick **one** filter policy across SDL_GPU and GLES — expose it as a canvas/app setting (`NEAREST` for pixel-art, `LINEAR` otherwise) instead of hard-coding a different choice per backend (`cute_graphics_sdlgpu.cpp:874` vs `cute_graphics_gles.cpp:736`).

---

8. Suggested future improvements

- **A DPI sample.** Add `samples/hidpi.c` demonstrating crisp text + shapes on a Retina/scaled display, and documenting `pixel_scale` vs `dpi_scale`. There is currently no HiDPI-focused sample.
- **A topics doc.** `docs/topics/` covers `application_window`, `drawing`, and `renderer` but has **no** DPI/HiDPI page. Add one explaining the point/pixel model and the retro-canvas escape hatch.
- **Per-monitor DPI stress test.** A CI/manual test that drags a window across monitors of different scales and asserts the canvas resizes and fonts re-rasterize.
- **Optional automatic canvas resize on window resize**, gated behind an app option, for apps that want the internal resolution to always track the window.
- **Signed-distance-field (SDF) font atlas** as a longer-term option: resolution-independent glyphs would make text sharp at any scale/zoom without per-DPI re-rasterization, and would also fix the "blurry when the camera zooms in" case that is the moral equivalent of the DPI problem.
- **Expose `cf_app_get_pixel_scale()` and a `CF_APP_OPTIONS_NO_HIGH_DPI_BIT`** so users can opt out of the forced `SDL_WINDOW_HIGH_PIXEL_DENSITY` (`cute_app.cpp:284`) when they explicitly want logical-resolution rendering.

---

9. Priority & effort

Priority	Fix	Effort	Risk	Payoff
P0	§7.1 Physical-resolution canvas + logical projection	Medium	Medium (touches camera/coordinate mapping)	Removes the entire upscale-blur class; sharpens *everything*
P0	§7.2 DPI-scaled glyph rasterization + cache key	Medium	Low	Crisp text
P1	§7.3 Physical-space anti-alias factor	Low	Low	Crisp shape edges
P1	§7.4 API/doc correction (`pixel_scale` , size semantics, touch)	Low	Low	Correctness & clarity
P2	§7.5 Consistent upscale filter policy	Low	Low	Backend parity
P2	§8 Sample + topic doc + SDF fonts	Medium–High	Low	Discoverability & long-term quality

Bottom line: the two P0 items (physical canvas + DPI-scaled glyphs) deliver ~90% of the visible improvement. They are coupled and should land together, because either one alone leaves the other's blur in place.

Appendix A — Key source references

Concern	File:line
High-DPI flag forced on	`src/cute_app.cpp:284`
`dpi_scale` from display scale	`src/cute_app.cpp:320`, `src/cute_input.cpp:558`
Offscreen canvas sized in logical units	`src/cute_app.cpp:155-167`, `:339`
Render-to-canvas then blit	`src/cute_app.cpp:527-536`
SDL_GPU stretch blit (NEAREST)	`src/cute_graphics_sdlgpu.cpp:842-878`
GLES stretch blit (LINEAR)	`src/cute_graphics_gles.cpp:717-741`
Logical resize handling	`src/cute_input.cpp:517-521`
DPI-change event	`src/cute_input.cpp:557-560`
Logical projection	`src/cute_draw.cpp:460`
Anti-alias factor (logical pixel)	`src/cute_draw.cpp:448-455`
Glyph rasterized at logical font size	`src/cute_draw.cpp:1936-1957`
Glyph cache key (no DPI)	`src/cute_draw.cpp:1845-1851`
Glyph placed 1:1 in world	`src/cute_draw.cpp:2759-2760`
"size in pixels" doc claim	`include/cute_app.h:367-388`
DPI API + "ignore dpi" advice	`include/cute_app.h:399-417`
Touch coords flagged for HiDPI	`src/cute_input.cpp:674-691`